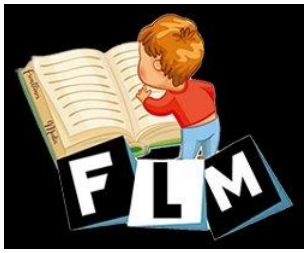

Frontlines Edutech

SQL Learning Guide

Views in SQL Server

#	Topic	Subtopics
1	What is a View?	Definition, how views work, simple example
2	CREATE VIEW	Basic syntax, creating a view with example
3	Updating Views	Updatable views, restrictions, WITH CHECK OPTION
4	Advantages & Limitations	When to use views, what views cannot do
5	Materialized View	Concept, difference from regular views, use cases
6	Interview Q & A	15 real-world fresher questions on Views
7	Practice Questions	20 exercises using flmstudents & flmcourses

https://www.frontlinesedutech.com

1. What is a View?

A View is a virtual table in SQL Server. It is created by writing a SELECT query, and that query is saved with a name. Whenever you use the view's name, SQL Server automatically runs the stored query and shows you the result — just like a real table.

Think of a View as a window into your data. You look through the window and see exactly what you want, without touching or moving the actual data stored in the original tables.

Definition

A View is a named, saved SELECT query stored in the database. It does not store data itself — it only stores the query logic. Every time you SELECT from a view, the underlying query runs fresh and returns current data.

Views are also called Virtual Tables because they behave like tables in queries but contain no physical data of their own.

How a View Works (Step by Step)

1. You write a SELECT query that pulls data from one or more tables.
2. You save that query as a View using CREATE VIEW.
3. Later, you can SELECT from the View just like a regular table.
4. SQL Server runs the stored query behind the scenes and returns the result.

Simple Example — Without a View

Suppose we always want to see active students from Hyderabad. Without a view, we write this query every time:

```
SELECT student_name, city, marks
FROM   flmstudents
WHERE  city = 'Hyderabad'
  AND  is_active = 1;
```

Same Example — Using a View

We create the view ONCE:

```
CREATE VIEW vw_hyd_active_students AS
SELECT student_name, city, marks
FROM   flmstudents
WHERE  city = 'Hyderabad'
```

```
  AND  is_active = 1;
```

Then we simply query the view whenever we need it:

```
SELECT * FROM vw_hyd_active_students;
```

Output

Result — Active students from Hyderabad

student_name	city	marks
Rahul Sharma	Hyderabad	88
Meena Das	Hyderabad	77
Rohit Patel	Hyderabad	68

Quick Tip: Think of a view as a saved question. You ask the question once (CREATE VIEW), and from then on you just say: give me the answer (SELECT * FROM view_name).

2. CREATE VIEW

CREATE VIEW is the SQL statement used to define a new view. You write it once, and SQL Server stores the query definition permanently in the database.

Syntax — Basic

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM   table_name
WHERE  condition;

-- To query the view later:
SELECT * FROM view_name;

-- To drop (delete) a view:
DROP VIEW view_name;

-- To modify an existing view:
ALTER VIEW view_name AS
SELECT ...  -- new query;
```

Rules for CREATE VIEW

- The view name must be unique in the database (just like a table name).
- The SELECT inside the view can include WHERE, JOIN, GROUP BY, ORDER BY, DISTINCT — almost anything.
- You cannot use ORDER BY in a view definition unless you also use TOP or OFFSET-FETCH.
- Column names in the view come from the SELECT list. You can alias them with AS.
- A view can be based on another view.

2.1 Example — Single Table View

Show only the student name, course, and marks from flmstudents:

```
CREATE VIEW vw_student_marks AS
SELECT student_name,
       course_id,
       marks
FROM   flmstudents;
```

```
SELECT * FROM vw_student_marks;
```

Output

Result — student_name, course_id, marks from flmstudents

student_name	course_id	marks
Rahul Sharma	C101	88
Priya Singh	C102	72
Amit Kumar	C101	95
Sneha Reddy	C103	80
...	...	...

2.2 Example — Multi-Table View (using JOIN)

Show student name together with the course name by joining both tables:

```
CREATE VIEW vw_student_course_details AS
SELECT s.student_name,
       s.city,
       s.marks,
       c.course_name,
       c.fees
FROM   flmstudents AS s
JOIN   flmcourses  AS c ON s.course_id = c.course_id;
```

```
SELECT * FROM vw_student_course_details;
```

Output

Result — student details joined with course details

student_name	city	marks	course_name	fees
Rahul Sharma	Hyderabad	88	SQL Fundamentals	5000
Priya Singh	Mumbai	72	Python Basics	6000
Amit Kumar	Delhi	95	SQL Fundamentals	5000
...	...	...	...	...

2.3 Example — View with Aggregation (GROUP BY)

Create a view that shows the average marks per course:

```
CREATE VIEW vw_avg_marks_per_course AS
SELECT course_id,
       COUNT(*)   AS total_students,
       AVG(marks) AS avg_marks,
       MAX(marks) AS top_score
FROM   flmstudents
GROUP BY course_id;
```

```
SELECT * FROM vw_avg_marks_per_course;
```

Output

Result — average marks per course

course_id	total_students	avg_marks	top_score
C101	3	86.67	95
C102	3	62.33	72
C103	2	85.50	91
C104	2	71.00	74

Quick Tip: Always prefix view names with 'vw_' so you can tell at a glance that it is a view, not a base table. Example: vw_active_students, vw_course_summary.

3. Updating Views

You can sometimes INSERT, UPDATE, or DELETE data through a view. When you do this, SQL Server actually modifies the underlying base table, not the view itself.

Definition

An updatable view is a view through which you can modify data in the underlying table. Not all views are updatable. SQL Server checks certain conditions before allowing DML (Data Manipulation Language) operations through a view.

3.1 When is a View Updatable?

A view is updatable (you can UPDATE / INSERT / DELETE through it) only when ALL of the following conditions are met:

- The view is based on a SINGLE base table (no JOINs for DML).
- The SELECT does NOT use DISTINCT.
- The SELECT does NOT use GROUP BY or HAVING.
- The SELECT does NOT use aggregate functions (SUM, COUNT, AVG, MAX, MIN).
- The SELECT does NOT use set operations (UNION, INTERSECT, EXCEPT).
- The columns being modified are NOT computed columns or expressions.
- The view does NOT use TOP with ORDER BY (for inserts/updates to be safe).

3.2 Example — Updatable View

```
-- Create a simple updatable view
CREATE VIEW vw_active_students AS
SELECT student_id, student_name, city, marks
FROM   flmstudents
WHERE  is_active = 1;

-- UPDATE through the view (modifies flmstudents directly)
UPDATE vw_active_students
SET    marks = 90
WHERE  student_id = 'S01';

-- INSERT through the view (adds a row to flmstudents)
INSERT INTO vw_active_students (student_id, student_name, city, marks)
VALUES ('S11', 'Aryan Roy', 'Pune', 78);

-- DELETE through the view (removes from flmstudents)
DELETE FROM vw_active_students
WHERE student_id = 'S11';
```

Note: When you INSERT or UPDATE through a view, the changes go into the BASE TABLE. If the inserted row does not satisfy the view's WHERE condition, it will NOT appear in the view — but it WILL be in the base table. Use WITH CHECK OPTION to prevent this.

3.3 WITH CHECK OPTION

WITH CHECK OPTION forces any INSERT or UPDATE done through the view to satisfy the view's WHERE condition. If the new data would NOT be visible in the view, SQL Server REJECTS the operation.

```
CREATE VIEW vw_active_students_safe AS
SELECT student_id, student_name, city, marks
FROM   flmstudents
WHERE  is_active = 1
WITH CHECK OPTION;

-- This INSERT will FAIL because is_active is not 1 (it is 0)
-- SQL Server rejects it to keep the view consistent
INSERT INTO vw_active_students_safe
  (student_id, student_name, city, marks, is_active)
VALUES ('S12', 'Test User', 'Delhi', 70, 0);
-- Error: The attempted insert or update failed because the
--        target view specifies WITH CHECK OPTION.
```

Quick Tip: Always use WITH CHECK OPTION on views that filter rows. It prevents 'invisible inserts' — rows that get added to the base table but never appear in the view.

3.4 ALTER VIEW — Modifying a View

To change an existing view's definition, use ALTER VIEW (not DROP + re-CREATE). This preserves permissions already granted on the view.

```
-- Original view
CREATE VIEW vw_student_marks AS
SELECT student_name, marks
FROM   flmstudents;

-- Modified view (adding city column)
ALTER VIEW vw_student_marks AS
SELECT student_name, city, marks
FROM   flmstudents;

-- Drop a view permanently
DROP VIEW vw_student_marks;
```

4. Advantages and Limitations of Views

4.1 Advantages

Views are one of the most useful features in SQL Server. Here is why developers and database administrators love them:

1. Simplicity — Complex queries become simple. Instead of writing a 10-line JOIN every time, you query a view in one line.

2. Security — You can give users access to a view without giving them access to the underlying table. Sensitive columns (like salary, password) can be hidden from the view.

3. Reusability — Write the logic once in the view definition and reuse it across many queries, stored procedures, and reports.

4. Consistency — If the business logic changes (e.g., active students are now those with is_active = 2), you update the view once — all queries using the view automatically get the updated logic.

5. Abstraction / Independence — Views hide the physical structure of the database. If a table is renamed or restructured, you can update the view and leave all the queries that use the view unchanged.

6. Logical Partitioning — You can split a large table into logical views (e.g., vw_students_hyd, vw_students_mum) for cleaner access patterns.

Example — Security Use Case

You have an Employee table with columns: emp_id, emp_name, department, salary, bank_account. You want HR staff to see names and departments but NOT salary or bank_account. Create a restricted view:

```
CREATE VIEW vw_employee_public AS
SELECT emp_id, emp_name, department
FROM   flmemployees;

-- Grant SELECT only on the view, NOT on the base table
GRANT SELECT ON vw_employee_public TO hr_user;
```

4.2 Limitations of Views

Views also have important restrictions you must know:

1. No Physical Data Storage — A regular (non-materialized) view does not store data. Every query on the view re-runs the underlying SELECT. On large tables this can be slow.

2. ORDER BY Restriction — You cannot use ORDER BY in a view definition unless you use TOP or OFFSET-FETCH. To sort, add ORDER BY in the outer query.

3. No Parameters — Views do not accept parameters. For parameterized queries, use stored procedures or table-valued functions instead.

4. DML Restrictions on Complex Views — You cannot INSERT, UPDATE, or DELETE through a view that has JOINs, GROUP BY, DISTINCT, or aggregate functions.

5. Cannot Index (Standard Views) — Regular views cannot be indexed directly. Only Indexed Views (a special SQL Server feature, similar to Materialized Views) support indexing.

6. Dependency Risks — If a base table is dropped or a column is renamed, the view will break. SQL Server does not automatically update views when tables change.

Advantages vs. Limitations — Quick Summary

Advantage	Limitation
Simplifies complex queries	Does not store data (re-runs every time)
Hides sensitive columns (security)	Cannot accept parameters
Reusable across many queries	ORDER BY not allowed without TOP
Keeps logic in one place (consistency)	DML restricted on complex views
Hides database structure (abstraction)	Breaks if base table structure changes
Logical partitioning of data	Cannot index standard views

5. Materialized View (Concept)

A Materialized View is an advanced type of view that physically stores the result of the query on disk — unlike a regular view that only stores the query definition.

Definition

A Materialized View (called an Indexed View in SQL Server) pre-computes and stores the query result as a real table on disk. The data is refreshed/updated when the base table changes. Because the result is already computed, queries are much faster.

5.1 Regular View vs Materialized View

Feature	Regular View	Materialized View (Indexed View)
Stores data physically?	No — only stores the query	Yes — stores the result on disk
Query speed	Depends on base table size	Very fast (pre-computed)
Data freshness	Always up-to-date (runs fresh)	Updated when base table changes
Storage required?	No extra storage	Yes — occupies disk space
Best used for	Simplicity, security, reuse	Large tables, heavy aggregations
SQL Server term	VIEW	INDEXED VIEW
Supported in MySQL?	Yes (regular views)	Via CREATE TABLE ... AS SELECT

5.2 Materialized Views in Other Databases

Database	Feature Name	Refresh Control
SQL Server	Indexed View (auto-refreshed)	Automatic (always fresh)
Oracle	Materialized View	Manual or scheduled refresh
PostgreSQL	Materialized View	REFRESH MATERIALIZED VIEW command
MySQL	No native materialized view	Simulate with CREATE TABLE + triggers

BigQuery	Materialized View	Automatic with configurable frequency

5.3 When to Use a Materialized View

- Reports and dashboards that aggregate millions of rows frequently.
- When query speed is critical and you can accept some storage cost.
- When the underlying data does not change very frequently.
- Data warehouses and analytics platforms where read performance matters most.

Quick Tip: In SQL Server, call it an Indexed View, not a Materialized View. The concept is the same, but the SQL Server term for interviews is Indexed View. Always mention that it requires SCHEMABINDING and a unique clustered index.

5. Real-Time Interview Questions & Answers

These questions are commonly asked in SQL Server interviews for fresher and junior developer roles. Study each carefully!

Views — Basic Concepts

Q: What is a View in SQL Server?

A: A View is a virtual table created from a saved SELECT query. It does not store data physically — it only stores the query definition. When you SELECT from a view, SQL Server runs the stored query and returns the result. Views are used for security, simplicity, and reusability.

Q: What is the difference between a Table and a View?

A: A Table physically stores data in rows and columns on disk. A View is a saved SELECT query that shows data from one or more tables — it does not store data itself. A Table takes disk space; a standard View takes almost none. You can query both in the same way, but DML (INSERT/UPDATE/DELETE) on Views has restrictions.

Q: Can you create a View from multiple tables?

A: Yes. You can JOIN two or more tables in the SELECT query inside the View definition. The View will show combined columns from all joined tables. However, you generally cannot use INSERT or

UPDATE through a multi-table View because SQL Server cannot determine which base table to modify.

Q: What is the naming convention recommended for Views?

A: Prefix the view name with 'vw_' so it is easy to distinguish from base tables. Examples: vw_active_students, vw_course_summary, vw_employee_public. This is a widely adopted convention in SQL Server development teams.

CREATE VIEW — Syntax & Rules

Q: Can you use ORDER BY inside a View definition?

A: Technically you can use ORDER BY only when it is combined with TOP or OFFSET-FETCH inside the view. A plain ORDER BY without TOP in a view definition is NOT allowed in SQL Server. To sort results from a view, add ORDER BY in the outer SELECT query that reads from the view.

Q: What is the difference between DROP VIEW and ALTER VIEW?

A: DROP VIEW permanently deletes the view from the database. ALTER VIEW changes the view's SELECT definition while keeping its name and any permissions that were granted on it. Use ALTER VIEW instead of DROP + re-CREATE so you don't lose permissions.

Q: Can a View be based on another View?

A: Yes — a View can SELECT from another View. This is called a nested view or layered view. It is valid but can hurt readability and debugging. Avoid deeply nested views; use CTEs or Indexed Views for complex scenarios instead.

Updating Views

Q: Can you always UPDATE data through a View?

A: No. A View is updatable only when it is based on a single table, has no DISTINCT, no GROUP BY, no aggregate functions (SUM, COUNT, etc.), and selects non-computed columns. Views with JOINs, GROUP BY, or aggregates are NOT directly updatable.

Q: What does WITH CHECK OPTION do in a View?

A: WITH CHECK OPTION ensures that any INSERT or UPDATE done through the view must satisfy the view's WHERE condition. If the new data would not be visible in the view, SQL Server rejects the operation. Without it, you could accidentally insert rows that 'disappear' from the view but exist in the base table.

Q: What happens to base table data when you UPDATE through a View?

A: The UPDATE modifies the actual rows in the underlying base table — not the view itself. The view is just a window; the real data always lives in the base table. After the UPDATE, querying the view will show the updated data because the view re-runs its SELECT against the now-updated base table.

Advantages, Limitations & Materialized Views

Q: What are the main advantages of using Views?

A: (1) Simplicity — complex queries become one-liners. (2) Security — hide sensitive columns from users. (3) Reusability — write logic once, use it everywhere. (4) Consistency — update logic in one place. (5) Abstraction — hide physical table structure from users.

Q: What is a limitation of a regular View in terms of performance?

A: A regular view does not cache or store results. Every time you SELECT from a view, SQL Server re-executes the full underlying query against the base tables. On large tables with complex JOINs or aggregations, this can be slow. Materialized Views (Indexed Views) solve this by pre-computing and storing the result.

Q: What is a Materialized View? How is it called in SQL Server?

A: A Materialized View physically stores the query result on disk. In SQL Server it is called an Indexed View. To create one, define the view WITH SCHEMABINDING and then create a UNIQUE CLUSTERED INDEX on it. SQL Server then stores the computed result and keeps it synchronized with the base table automatically.

Q: What is the difference between a View and a CTE (Common Table Expression)?

A: A CTE is temporary — it exists only for the duration of a single query. A View is permanent — it is stored in the database and can be reused across sessions and queries. Use a CTE for one-off complex query logic within a single statement. Use a View for logic that is reused frequently or needs to be shared with other users or processes.

7. Practice Questions

All practice questions use the same two tables : flmstudents and flmcourses. Write your queries in SQL Server Management Studio (SSMS) and verify the output.

Reference Tables — Use These for All Practice Questions

Table 1: flmstudents

student_id	student_name	city	course_id	marks	is_active
S01	Rahul Sharma	Hyderabad	C101	88	1
S02	Priya Singh	Mumbai	C102	72	1
S03	Amit Kumar	Delhi	C101	95	1
S04	Sneha Reddy	Mumbai	C103	80	0
S05	Ravi Verma	Bangalore	C102	60	1
S06	Meena Das	Hyderabad	C101	77	1
S07	Arjun Nair	Delhi	C103	91	0
S08	Kavya Rao	Bangalore	C102	55	1
S09	Rohit Patel	Hyderabad	C104	68	1
S10	Divya Mehta	Mumbai	C104	74	0

Table 2: flmcourses

course_id	course_name	instructor	fees	city
C101	SQL Fundamentals	Ravi Kumar	5000	Hyderabad
C102	Python Basics	Priya Sharma	6000	Mumbai
C103	Data Analysis	Amit Verma	7500	Delhi
C104	Excel for Data	Sneha Roy	4000	Bangalore
C105	Power BI	Kiran Das	8000	Hyderabad

CREATE VIEW — Questions 1 to 5

1 Create a view named vw_all_students that shows all columns from flmstudents.

2 Create a view named vw_student_city_marks that shows only student_name, city, and marks from flmstudents.

3	Create a view named vw_mumbai_students that shows all students from Mumbai. Then SELECT all columns from the view.
4	Create a view named vw_course_details that joins flmstudents and flmcourses on course_id. Show student_name, course_name, and fees.
5	Create a view named vw_high_scorers that shows students with marks >= 80. Include student_name and marks.

Updating Views — Questions 6 to 10

6	Create a simple view vw_active_students showing all active students (is_active = 1). Then UPDATE marks to 85 for student S05 through the view.
7	Create the view vw_active_students WITH CHECK OPTION. Try to INSERT a student with is_active = 0 through the view. What happens?
8	Use ALTER VIEW to add the course_id column to vw_student_city_marks (created in Q2).
9	Create a view vw_delhi_students showing Delhi students. INSERT a new student (S11, 'Vikram Rao', 'Delhi', 'C101', 82, 1) through the view. Then verify it appears in both the view and the base table.
10	DROP the view vw_mumbai_students. Then verify it no longer exists by querying INFORMATION_SCHEMA.VIEWS.

Views with Aggregation — Questions 11 to 15

11	Create a view named vw_course_enrollment_count that shows each course_id and the number of students enrolled (total_students). Use GROUP BY.
12	Create a view vw_city_marks_summary that shows each city, average marks (avg_marks), highest marks (max_marks), and lowest marks (min_marks).

13	Query vw_city_marks_summary (created in Q12) to show only cities where the average marks are above 75.
14	Create a view vw_expensive_courses that shows course_name, instructor, and fees for courses costing more than 5000. Then SELECT from the view sorted by fees descending.
15	Create a view vw_student_full_info by joining flmstudents and flmcourses. Show student_name, city, marks, course_name, instructor, and fees. Then find the most expensive course a student from Hyderabad is enrolled in.

Advanced Views — Questions 16 to 20

16	Create a view vw_public_student_info that shows only student_name and city (hiding marks and is_active). Explain why this is useful from a security perspective.
17	CHALLENGE: Create a view that uses a subquery inside the WHERE clause. Show students whose marks are above the overall average marks in flmstudents.
18	CHALLENGE: Can you use ORDER BY in a view definition without TOP? Try writing a view with ORDER BY marks DESC. What error does SQL Server give? Rewrite it using TOP 100 PERCENT.
19	Create a view vw_inactive_students showing students where is_active = 0. Can you UPDATE is_active to 1 through this view? Will WITH CHECK OPTION allow or block this update? Explain why.
20	INDEXED VIEW: Write the steps (CREATE VIEW WITH SCHEMABINDING + CREATE UNIQUE CLUSTERED INDEX) to create an Indexed (Materialized) View named vw_indexed_course_summary that shows course_id and COUNT_BIG(*) as student_count from dbo.flmstudents.

Quick Tip: After writing each query, also try: SELECT * FROM INFORMATION_SCHEMA.VIEWS; — This lists all views in your database and is a great way to confirm your views were created correctly.